

Agent OS — Worker Agent Library v1

Status: Working specification. Source of truth for what each agent does, how it gets invoked, and what it produces. **Date:** 28 May 2026 **Owner:** Aidan **Scope:** General-purpose agents for small business owners. Not vertical-specific. Industry packs come later by curating subsets of this library and tuning their tone.

How to read this document

This spec defines the **v1 worker agent library** for Agent OS — the set of specialist agents the orchestrator routes to. It is written to be implementable directly by a coding agent.

Agents are organized into **functional buckets** that mirror how a small business owner thinks about their work: Customer Service, Sales, Marketing, Scheduling & Operations, Finance, Reputation, Reporting & Insight, and System. Within each bucket, every agent has the same schema (see §2). The orchestrator routes owner asks to whichever specialist best matches; the owner never has to know about buckets — they're a maintenance and prioritization construct for the team.

Substrate dependency — read this first. Before any new agent gets built or any existing agent gets refactored, **the business profile data captured at signup must be wired into every worker's prompt**. Without that, every agent on this list will produce drafts containing `[Shop Name]` / `[Your Name]` / `[Phone]` placeholders. The substrate fix is the single highest-leverage prerequisite — see Phase 1 of the product plan. Treat it as a hard dependency, not a parallel track.

1. Library at a glance

Bucket	Agent	Channel	Status	Build priority
Customer Service	Customer Question	widget / email	Existing — refactor + fix high-sev bug	P1
	Complaint Handler	widget / email	New	P3

Bucket	Agent	Channel	Status	Build priority
Sales	Lead Nurture	sequence	Existing — refactor	P1
	Quote Follow-up	sequence	New (migration)	P2
Marketing	Campaign	email + SMS	Existing — refactor	P1
	Content Writer	report (long-form)	New (migration)	P2
	Social Post	post	New (migration)	P2
	SEO Recommendations	report	New (workaround)	P3
Scheduling & Operations	Booking	SMS / email	Existing — refactor	P1
	Appointment Reminder	SMS	New (Phase 4 trigger-driven)	P4
Finance	Invoice Reminder	email / SMS	New	P2
	Payment Follow-up	sequence	New	P3
	Quote Generator	email / report	New	P2
Reputation	Review Request	SMS / email	New (migration)	P2
	AI Visibility (stub)	report	New (stub)	P3
Reporting & Insight	Weekly Briefing	report (Mon)	New (replaces Analytics)	P2
System (internal)	Lead Triage	internal	New (Phase 4 trigger-driven)	P4

Bucket	Agent	Channel	Status	Build priority
	Generalist	report (free-form)	Existing — refactor as honest fallback	P1

Build priority key:

- **P1** — refactor existing agents into the new schema. Fix QA bugs. ~weeks 7–9.
- **P2** — clean migration of paid features into the agent paradigm. ~weeks 10–13.
- **P3** — workarounds and second-pass agents that need slightly more design (Complaint, SEO Reco, AI Visibility, Payment Follow-up). ~weeks 13–16.
- **P4** — depend on Phase 4 triggers and/or real-action capabilities. Defer until those ship.

A list of further agents to build later (No-Show Recovery, Lapsed Customer Reactivation, FAQ Builder, Page Editor, Seasonal Campaign, Pricing Question Handler, Onboarding Coach) is in §10.

2. The agent schema

Every agent in this library conforms to this schema. The coding agent should build a registry that enforces it.

agent_id: snake_case_unique_id

display_name: "Human-Readable Name"

bucket: customer_service | sales | marketing | scheduling_ops | finance | reputation | reporting | system

status: existing | new | workaround | stub

build_priority: P1 | P2 | P3 | P4

purpose: one sentence

routes_here_when:

- description of owner ask patterns

- description of event patterns (for triggered agents)

channel: sms | email | sequence | report | post | widget_reply | internal

inputs:

from_owner: # extracted by orchestrator from natural-language ask

- name: field_name

type: string | number | date | array

required: true | false

description: ...

from_shared_context:

- business_profile # always required for v1
- widget_history # optional, declare if used
- pipeline_state # optional, declare if used
- agent_run_history # optional, declare if used

tool_dependencies: # external tools the agent calls. drafts-only agents declare none.

- none | google_calendar | gmail_send | twilio_sms | seo_check | invoice_lookup

permission_scope:

default: drafts_only

configurable_phase_4:

- require_owner_approval: true | false

- send_caps:

per_day: N

per_month: N

- recipient_filter: existing_customers_only | any | custom_list

triggers_supported:

- manual # always
- scheduled: cron expression(s) supported
- event_based: list of events the agent can subscribe to

outputs:

title_format: ...

body_format: ...

metadata:

- key: value pairs the orchestrator captures with the draft

reasoning_trace_steps: # what the trace SHOULD honestly show

- step_1_name: description (only emit if the underlying load actually returned data)
- ...

example_interactions:

- owner_ask: "..."

expected_route: agent_id

expected_output_excerpt: "..."

Three rules the registry must enforce, regardless of bucket:

1. **No agent may emit a "loaded" trace step for a resource that returned empty data.**
When a load returns nothing, the trace must say "no X yet — using fallback" or skip the step entirely. This is the honest-trace rule from the QA report.
2. **No agent may emit bracketed placeholders ([Shop Name], [Your Name], etc.)** for fields that exist in **business_profile**. If the field is missing from the profile, the agent must say so to the orchestrator — never to the customer.

3. **Every agent declares its channel.** SMS-channel agents must produce plain text (no markdown). Email-channel agents may use markdown that gets converted at send time. Owner-only reports may use full markdown.
-

3. Customer Service bucket

Agents that respond to inbound customer communication — questions, complaints, support requests.

3.1 Customer Question `customer_question`

Bucket: `customer_service` · **Status:** Existing (refactor + fix high-sev bug) · **Priority:** P1

Purpose: Drafts written answers to customer questions about hours, services, pricing, policies, or products.

Routes here when:

- Owner pastes a customer question and asks for a reply
- (Phase 4) Event: new widget conversation classified as "question" intent

Channel: `widget_reply` (default); `email` or `sms` if requested.

Inputs

- From owner ask: `customer_question` (verbatim, required), `customer_name` (optional), `customer_context` (optional)
- From shared context: `business_profile`, `widget_history` (search relevant prior conversations), `kb` (owner-curated FAQ if any)

Tool dependencies: None.

Outputs

- Title: `Reply to {customer_name or "lead"} - {topic}`
- Body: 1–3 short paragraphs, channel-appropriate format. **Never** contains internal back-channel content (see critical bug below).

Critical QA bug — must fix during refactor. Today, when the business KB is empty, this agent produces a customer-facing draft whose body is actually an internal request to the owner: `"I`

need your business profile and knowledge base to draft an accurate response for you. Could you please share..." Required new behaviour:

- If the KB lacks the needed info → produce a **safe holding reply** that acknowledges the customer, asks one clarifying question if appropriate, and promises follow-up. Never invent facts. Never expose internal status to the customer.
- Surface the gap **separately in the orchestrator chat**: *"Heads up — I don't have any info on hybrid service in your KB. Want to add it now so I can sharpen this reply?"*

Example

Owner ask: "A new lead asked through the widget: 'Do you guys handle hybrids? I have a 2018 Prius and the battery feels weak.' Draft a response."

Expected output (when KB lacks hybrid info):

Hi! Thanks for reaching out to Sunset Mobile Detailing. Hybrid battery work is something I want to confirm with our technician before quoting you — let me check and get back to you within the hour. In the meantime, is the warning light on, and roughly how old is the battery? — Alex

3.2 Complaint Handler **complaint_handler**

Bucket: customer_service · **Status:** New · **Priority:** P3

Purpose: When a customer message contains complaint language, drafts a sensitive, empathetic response AND raises a flag in the orchestrator chat so the owner sees it personally. Treats complaints as higher-stakes than generic customer questions because the cost of getting one wrong is high.

Routes here when:

- Owner asks for help responding to a complaint
- (Phase 4) Lead Triage classifies a widget message as **complaint** intent

Channel: **widget_reply** or **email**

Inputs

- From owner ask: **customer_name**, **complaint_text** (verbatim), **incident_date**, **prior_history** (optional)

- From shared context: `business_profile`, `pipeline_state` for this customer

Tool dependencies: None.

Permission scope: **never auto-send**. Always requires owner approval, even after Phase 4 ships, even if the owner has enabled auto-send for other agents. Hardcoded.

Outputs

- Title: `Complaint reply – {customer_name}, {topic}` (flagged red in the task list)
 - Body: empathetic acknowledgment, ownership of the issue, one concrete next step. Never minimises. Never makes promises the business can't keep.
-

4. Sales bucket

Agents that move leads through the pipeline — from first interest to booked appointment or signed engagement.

4.1 Lead Nurture `lead_nurture`

Bucket: sales · **Status:** Existing (refactor) · **Priority:** P1

Purpose: Drafts warm follow-up messages or 2–3 touch sequences to re-engage prospects who haven't moved forward.

Routes here when:

- Owner asks for a follow-up sequence for a specific lead
- (Phase 4) Trigger: lead stale N days

Channel: `sequence` (multi-touch, mixed channels by default)

Inputs

- From owner ask: `customer_name`, `subject` (e.g. "consultation inquiry"), `last_contact_date`, `current_status`, `tone_hint` (default "warm, not pushy")
- From shared context: `business_profile`, `pipeline_state`, `agent_run_history` (avoid repeating earlier outreach)

Tool dependencies: None at draft stage.

Triggers supported: manual; Phase 4 adds scheduled and event-based.

Outputs

- Title: {N}-touch follow-up sequence – {customer_name}, {subject}
- Body: clearly delimited sections per touch, using **relative dates** (Touch 1 – Today (Text), Touch 2 – +5 days (Email), etc.). Each touch shows its target channel.

QA bugs to fix while refactoring

- Use relative dates (Today / +5 / +14) not Day 1 / Day 5 framing
 - Internal consistency: if Touch 2 body says "a little over a week ago", the date label must match
 - Use real business name in signoffs, not [Shop Name]
-

4.2 Quote Follow-up quote_follow_up

Bucket: sales · **Status:** New (migration) · **Priority:** P2

Purpose: Specialised lead-nurture variant tuned for unresponded quotes. Carries the quote details (amount, scope) and uses quote-specific framing — softer than chasing payment, sharper than generic nurture.

Routes here when:

- Owner asks to follow up on a specific quote that hasn't been booked
- (Phase 4) Event trigger: quote sent → +3 days, +7 days, +14 days if no booking

Channel: sequence (3-touch by default) or sms (single touch)

Inputs

- From owner ask: customer_name, quote_amount, quote_scope, quote_date, touch_count (default 3), tone_hint
- From shared context: business_profile, pipeline_state

Outputs

- Title: Quote follow-up – {customer_name}, \${quote_amount} {quote_scope}

- Body: sequence structure with quote-specific framing in each touch

Overlap with Lead Nurture: Quote Follow-up handles the specific case where there's a known quote dollar amount and scope. Lead Nurture handles broader stale-lead scenarios. The orchestrator routes here only when the owner's ask contains a dollar amount + the word "quote" (or equivalent).

5. Marketing bucket

Agents that produce outbound communication to customers and prospects — campaigns, content, social, search visibility.

5.1 Campaign **campaign**

Bucket: marketing · **Status:** Existing (refactor) · **Priority:** P1

Purpose: Drafts marketing campaigns — promotions, announcements, seasonal offers, email blasts, and short SMS campaigns.

Routes here when:

- Owner asks for a campaign / email blast / promo announcement
- Owner asks for subject line + body for a marketing send

Channel: **email** (primary), **sms** (short-form). Social variants delegate to Social Post.

Inputs

- From owner ask: **campaign_name**, **audience** (existing customers / new / all), **offer_details** (price, dates, eligibility), **tone_hint**, **length_hint**, **desired_deliverables** (default: subject + preheader + body)
- From shared context: **business_profile**, **pipeline_state** (for audience sizing)

Tool dependencies: None at draft stage. Phase 4 adds **gmail_send** and **twilio_sms**.

Permission scope (Phase 4):

- **recipient_filter**: **existing_customers_only** is the safe default
- **send_caps**: hard cap on broadcast sends per day

Outputs

- Title: `Email blast – {campaign_name} ({offer_summary})`
- Body: subject line (≤30 char if mobile-optimized), preheader, email body. Optionally include social variant **only if** owner asked for one.

QA bugs to fix while refactoring

- Front-load price/\$ in subject line (mobile inboxes clip after ~30 chars)
 - Respect "keep it short" — don't add unrequested deliverables
 - Make emoji density a parameter, default lower than today
 - Real business name in signoff
-

5.2 Content Writer `content_writer`

Bucket: marketing · **Status:** New (migration) · **Priority:** P2

Purpose: Drafts longer-form content — blog posts, website copy, hosted-page sections, FAQs, About Us paragraphs.

Routes here when:

- Owner asks for a blog post, article, web copy, long-form deliverable
- Owner asks to draft FAQ entries, About Us copy, service descriptions

Channel: `report` (long-form, owner-facing)

Inputs

- From owner ask: `topic`, `format` (blog / FAQ / about / service description), `target_length`, `tone_hint`, `seo_keywords` (optional), `audience`
- From shared context: `business_profile`

Tool dependencies: None.

Outputs

- Title: `{format} – {topic}`
 - Body: structured per format
-

5.3 Social Post `social_post`

Bucket: marketing · **Status:** New (migration) · **Priority:** P2

Purpose: Drafts a single social media post or short thread. Platform-aware.

Routes here when:

- Owner asks for a social post or caption
- Campaign agent delegates the social variant of a campaign here

Channel: `post` (platform-specific format)

Inputs

- From owner ask: `topic`, `platform` (default: Facebook), `tone_hint`, `length_hint`, `hashtags_wanted` (default false), `cta` (optional)
- From shared context: `business_profile`

Tool dependencies: None at draft stage. Phase 4 may add a posting tool (defer).

Outputs

- Title: `Social post ({platform}) - {topic}`
 - Body: platform-appropriate length (FB/IG ~200–500 chars, LinkedIn 1–3 paragraphs, X ≤280)
-

5.4 SEO Recommendations `seo_recommendations`

Bucket: marketing · **Status:** New (workaround) · **Priority:** P3

Purpose: Returns generic SEO best-practice recommendations for the owner's URL, plus basic on-page checks (title tag length, meta description, heading structure, mobile viewport, image alt text). **Does not perform a full crawl.** Full crawler is Phase 5+ infrastructure.

Routes here when:

- Owner asks for an SEO audit / SEO check / SEO recommendations

Channel: `report`

Inputs

- From owner ask: `url` (optional, defaults to `business_profile.website`)
- From shared context: `business_profile`

Tool dependencies: `seo_check` (lightweight on-page checker — fetches URL, parses HTML, returns findings). Build as part of this agent's implementation.

Outputs

- Title: `SEO recommendations - {domain}`
- Body: structured report covering on-page basics, content recommendations, **and an explicit list of what's NOT being checked yet** (backlinks, rankings, competitor analysis). Sets honest expectations.

Marketing note: sell as "SEO recommendations," not "SEO audit."

6. Scheduling & Operations bucket

Agents that manage the owner's time and appointments — booking, confirmations, reminders, day-to-day operational flow.

6.1 Booking `booking`

Bucket: `scheduling_ops` · **Status:** Existing (refactor) · **Priority:** P1

Purpose: Drafts appointment booking, rescheduling, cancellation, and confirmation messages addressed to a specific customer.

Routes here when:

- Owner asks to text/email a customer about an appointment slot
- Owner asks to confirm, reschedule, or cancel an existing appointment
- (Phase 4) Event: new widget conversation expressing booking intent

Channel: `sms` (default); `email` if requested; `sequence` (text + email reminder) is Phase 4.

Inputs

- From owner ask: `customer_name`, `subject` (e.g. "consultation"), `requested_day` (optional), `offered_slot`, `service_type` (optional), `notes` (optional)

- From shared context: `business_profile`

Tool dependencies: None at draft stage. Phase 4 adds `google_calendar` (write).

Permission scope (Phase 4):

- `require_owner_approval`: true by default
- `send_caps`: 30 SMS/day default

Triggers supported: manual; Phase 4 adds scheduled (day-before reminders — see Appointment Reminder).

Outputs

- Title: `SMS to {customer_name} - {day} {time} booking`
- Body: plain text, ≤ 280 chars. No markdown. Standard SMS opt-out language where applicable.

QA bugs to fix while refactoring

- Strip markdown from SMS outputs
- Resolve "would that work for you?" + "consider this your confirmation" contradiction — pick one frame
- Don't fabricate scheduling state ("fully booked tomorrow" when the owner said "earliest is Thursday")

6.2 Appointment Reminder `appointment_reminder`

Bucket: `scheduling_ops` · **Status:** New · **Priority:** P4 (depends on Phase 4 triggers + tools)

Purpose: Sends day-before SMS reminders for upcoming appointments.

Routes here when:

- Owner asks to send reminders for tomorrow's appointments
- (Phase 4) Scheduled trigger: daily at owner-configured time

Channel: `sms`

Inputs

- From shared context: appointments scheduled for tomorrow (from Calendar tool), `business_profile`

Tool dependencies: `google_calendar` (read), `twilio_sms` (send, Phase 4)

Outputs

- One short SMS per upcoming appointment

Permission scope:

- `recipient_filter`: only customers with appointments scheduled
 - `send_caps`: 1 reminder per appointment (hardcoded)
 - After Phase 4 trust ramps, can become auto-send (low risk; high routine value)
-

7. Finance bucket

Agents that handle money-related communication — quotes, invoices, payment collection. Drafts-only at v1. Real send and integrations with billing systems come later.

7.1 Quote Generator `quote_generator`

Bucket: finance · **Status:** New · **Priority:** P2

Purpose: Drafts a quote document or email for a service the owner is proposing. Captures line items, prices, terms, validity period.

Routes here when:

- Owner asks to draft a quote / estimate / proposal for a customer

Channel: `email` (default) or `report` (PDF-style document for owner to send via their own channel)

Inputs

- From owner ask: `customer_name`, `service_items` (array of {description, price, quantity}), `terms` (e.g. "valid 30 days, 50% deposit"), `validity_days` (default 30), `notes` (optional)
- From shared context: `business_profile`

Tool dependencies: None at draft stage. Phase 5+ could integrate with Stripe/QuickBooks for live quote-to-invoice flow.

Outputs

- Title: `Quote - {customer_name}, ${total}`
- Body: standard quote format — header with business profile + customer name, itemized table, total, terms, validity, signoff. Use real business name + contact info; never `[Shop Name]`.

Permission scope:

- `require_owner_approval`: true always — quotes commit to a price
- Phase 4 may allow direct send, but never auto-generate-and-send without explicit owner review

Example

Owner ask: "Draft a quote for Mike Johnson — full brake job (pads + rotors) on his 2019 F-150, parts \$620, labor \$480, terms net 15."

Expected output: itemized quote totaling \$1,100 with Sunset Mobile Detailing branding, 30-day validity by default.

7.2 Invoice Reminder `invoice_reminder`

Bucket: finance · **Status:** New · **Priority:** P2

Purpose: Drafts a polite reminder for an unpaid invoice. First-touch tone — assumes the customer forgot, not that they're avoiding payment.

Routes here when:

- Owner asks to follow up on an unpaid invoice (typically 1–14 days overdue)

Channel: `email` (default) or `sms`

Inputs

- From owner ask: `customer_name`, `invoice_amount`, `invoice_date`, `days_overdue`, `payment_method_options` (optional)

- From shared context: `business_profile`

Tool dependencies: None at draft stage. Phase 5+ could integrate with billing systems to pull invoice data automatically.

Permission scope:

- `require_owner_approval`: true always
- `send_caps`: 1 reminder per invoice per 7 days (hardcoded; prevents spam)

Outputs

- Title: `Invoice reminder - {customer_name}, ${amount}`
- Body: friendly, brief. Mentions the invoice date and amount. Offers payment options if provided. No threatening language at first touch.

Example

Owner ask: "Send Mike Johnson a reminder for invoice #1042 — \$1,100, 8 days overdue."

Expected output:

Hi Mike, hope you're doing well! Just a friendly reminder about invoice #1042 from May 19 (\$1,100). If it's already in process, please ignore this — otherwise let me know if you'd like to pay by [methods]. Thanks! — Alex, Sunset Mobile Detailing

7.3 Payment Follow-up `payment_follow_up`

Bucket: finance · **Status:** New · **Priority:** P3

Purpose: Drafts an escalating payment-chase sequence for invoices that didn't respond to the first reminder. 3-touch by default with firmer (but still professional) tone progression.

Routes here when:

- Owner asks for an escalation sequence on an overdue invoice
- (Phase 4) Trigger: invoice 14+ days overdue with no response to first reminder

Channel: `sequence`

Inputs

- From owner ask: `customer_name`, `invoice_amount`, `invoice_date`, `days_overdue`, `prior_touces` (so we don't repeat), `escalation_level` (1=firm friendly, 2=formal, 3=final notice)
- From shared context: `business_profile`, `pipeline_state` (customer history)

Tool dependencies: None at draft stage.

Permission scope:

- `require_owner_approval`: **always** — payment escalation is too high-stakes to ever auto-send

Outputs

- Title: `Payment follow-up - {customer_name}, ${amount}, level {escalation_level}`
- Body: structured per escalation level. Level 1 = warm reminder. Level 2 = formal notice mentioning past-due status. Level 3 = final notice mentioning next steps (collections referral, lien filing, etc. — but **only** general language; never specific legal threats without owner customization).

Hard rules:

- Never use threatening or accusatory language without explicit owner direction
- Never reference legal action specifics — only general "next steps will be considered"
- Always include a way for the customer to resolve quickly (payment link placeholder if available, contact for payment plan)

8. Reputation bucket

Agents that manage the business's online presence — reviews and AI search visibility.

8.1 Review Request `review_request`

Bucket: reputation · **Status:** New (migration) · **Priority:** P2

Purpose: Drafts post-service review-request messages.

Routes here when:

- Owner asks to send a review request to a specific recent customer
- (Phase 4) Event trigger: appointment marked complete in calendar → fires 24 hours later

Channel: `sms` (default), `email`

Inputs

- From owner ask: `customer_name`, `service_completed`, `service_date`, `platform_preference` (Google / Yelp / Facebook — default Google)
- From shared context: `business_profile` (must include `review_link_google` etc. once owner configures)

Tool dependencies: None at draft stage.

Permission scope (Phase 4):

- `recipient_filter`: only customers with completed service
- `send_caps`: 1 per customer per 90 days (hardcoded; avoids spam)

Outputs

- Title: `SMS to {customer_name} – review request for {service}`
- Body: short, warm, asks for the review with a link

8.2 AI Visibility (stub) `ai_visibility_stub`

Bucket: reputation · **Status:** New (stub) · **Priority:** P3

Purpose: Acknowledges the AI Visibility feature exists, returns a placeholder report explaining we're tracking how AI search engines describe the business, and notes that full results come in a later release. Captures interested-owner signal for the v2 beta.

Routes here when:

- Owner asks about AI visibility / GEO score / how ChatGPT or LLMs see their business

Channel: `report`

Inputs

- From owner ask: `query` (optional)

- From shared context: `business_profile`

Tool dependencies: None for the stub. Real version (Phase 5+) requires partnership or build-out.

Outputs

- Title: `AI Visibility - {business_name}`
- Body: short placeholder honestly explaining state. Tags interested owner for v2 beta.

Marketing note: do not price upgrade tiers on this feature until the real version ships.

9. Reporting & Insight bucket

Agents that summarize what's happening across the business — the owner's situational awareness.

9.1 Weekly Briefing `weekly_briefing`

Bucket: reporting · **Status:** New (replaces Analytics dashboard) · **Priority:** P2

Purpose: Replaces the Analytics dashboard. Produces a written summary every Monday morning of what happened last week and what's coming up.

Routes here when:

- Owner asks "what happened last week" / "give me a summary" / "run my weekly briefing"
- Phase 4 scheduled trigger: every Monday 7am owner-local time

Channel: `report` (in-app, with optional email digest in Phase 4)

Inputs

- From owner ask: `period` (default "last 7 days"), `focus` (default "all")
- From shared context: `business_profile`, `widget_history`, `pipeline_state`, `agent_run_history`

Tool dependencies: None.

Triggers supported: manual + scheduled (default `0 7 * * MON` in owner's timezone)

Outputs

- Title: **Weekly Briefing – week of {date}**
- Body: structured report. Sections (only included if non-empty):
 - **Conversations:** N widget chats, top topics, intent breakdown
 - **Leads:** new this week, pipeline movement, top stale leads to follow up
 - **Appointments:** booked, completed, no-shows
 - **Drafts & sends:** what agents produced, what owner approved, what was sent
 - **Finance:** outstanding invoices, unpaid quotes (data depends on Finance agent run history)
 - **What's coming:** scheduled triggers / agents firing this week
 - **Owner attention needed:** anything flagged (complaints, wishlist items, missing info)

Critical rule: if a section has no data, **omit it**. Don't produce a "Conversations: none this week" line — drop the section entirely. The briefing should feel substantive when there is activity and quiet when there isn't.

10. System bucket (internal-only agents)

Not user-facing as standalone capabilities. Either internal infrastructure (Lead Triage routes events to other agents) or fallback behavior (Generalist catches what doesn't match anything else).

10.1 Lead Triage **lead_triage**

Bucket: system · **Status:** New · **Priority:** P4 (depends on Phase 4 event triggers)

Purpose: Internal meta-agent. Fires when a new widget conversation closes. Classifies the conversation's intent (booking / question / complaint / spam / sales pitch / qualified lead). Slots the conversation into the right place in the pipeline. Optionally fires the matching specialist (Customer Question for questions, Booking for booking intent, Complaint Handler for complaints).

Channel: **internal** — produces no owner-facing draft, but may fire other agents that do.

Inputs: widget conversation transcript + contact info

Tool dependencies: None.

Triggers supported: event-based only (`widget_conversation_closed`)

Permission scope: can fire other agents in `draft_only` mode regardless of owner trust level for those agents. Owner-facing escalation only.

10.2 Generalist `generalist`

Bucket: system · **Status:** Existing (refactor as honest fallback) · **Priority:** P1

Purpose: True fallback. Handles requests that don't fit any other specialist. Drafts documents, summaries, plans, outreach copy — anything that's writing work but not a known specialist.

Routes here when:

- Orchestrator's classifier returns low confidence on all other specialists
- Owner explicitly asks for something open-ended ("write me a list of ideas to...")

Channel: `report` (owner-facing free-form)

Inputs

- From owner ask: `request` (verbatim), `format_hint` (memo / list / paragraph)
- From shared context: `business_profile` (always)

Tool dependencies: None.

Outputs

- Title: `{format_hint or "Draft"} - {short summary of request}`
- Body: free-form, owner-facing

Critical refactor — solve the "empty draft" failure mode.

The current generalist produces empty placeholder drafts that just echo the owner's request back. New behavior:

1. If the LLM is available and produces content: return that content as the draft.
2. If the LLM is unavailable (credits, outage, downstream error): **do not produce a draft.** Instead surface in the orchestrator chat: "Service is temporarily unavailable — try again in a few minutes." No draft, no Approve button.

3. If the classifier's top specialist confidence was actually close (say >0.4) but routed here as fallback: **offer the close match before generating** — "I'm not totally sure this fits, but it's close to the {nearest_specialist} agent. Want me to try that, or proceed with a general response?"
4. **Wishlist capture lives here.** When confidence is low across the board, the orchestrator acknowledges in chat, offers the nearest match, and captures the request as a wishlist item with the owner's vertical, considered specialists, and the verbatim ask.

Triggers supported: manual only

11. Orchestrator routing rules

The orchestrator is the classifier that picks which agent handles an owner ask. It runs on Haiku (per the cost discipline plan). Rules:

1. **Confidence threshold:** if top match confidence < 0.5 , route to **generalist** with wishlist capture and offer nearest match.
2. **Confidence resolution:** if top two matches are within 0.1 of each other, show both to the owner and ask: "Sounds like booking or a customer question — which is it?"
3. **Owner override:** the routing decision is always visible. The owner can re-route a task to a different specialist before approving.
4. **Channel inference:** if the owner says "text", route to an SMS-channel agent; "email", to email; if ambiguous, ask.
5. **Specialty preference:** the specialised agent (e.g. Quote Follow-up) outranks the generic (Lead Nurture) when the owner's ask contains the specialty triggers (a dollar amount + "quote").
6. **Sensitive routing:** complaint-language detection short-circuits Customer Question and routes to Complaint Handler.
7. **Bucket awareness:** the orchestrator can answer "what marketing agents do you have?" or "what can you do in finance?" by listing agents per bucket. Buckets are user-discoverable, not hidden.

Routing logs (input $\rightarrow$ chosen agent $\rightarrow$ confidence $\rightarrow$ owner accepted/changed) accumulate from day one. After 1000+ logged decisions, fine-tune a small classifier on real owner data.

12. What we are NOT building in v1

This list exists to prevent scope creep.

- **Multi-agent workflow chaining within a single owner ask** ("plan my Q4 marketing including a campaign email AND social posts AND a follow-up sequence"). v1 handles these as sequential separate asks. Chaining is Phase 5+.
 - **Owner-created custom agents**. v1 is a fixed library. Custom agents come after the registry has proven robust on the fixed set.
 - **Full SEO crawler**. Workaround agent only. Real version Phase 5+.
 - **Real LLM-search visibility tracking**. Stub only.
 - **Voice channel (phone)**. Defer.
 - **Multi-language drafts**. v1 is English only.
 - **Image generation (logos, post graphics)**. Defer.
 - **Billing-system integration (QuickBooks, Stripe, FreshBooks)**. Finance agents are drafts-only in v1; integration is Phase 5+.
-

13. Future agents (not in v1, prioritized for later)

In rough priority order. Full specs get written before each one is built.

1. **No-Show Recovery** (scheduling_ops) — triggered when an appointment is marked no-show. Drafts re-engagement message.
 2. **Lapsed Customer Reactivation** (sales) — owner asks (or scheduled trigger) for outreach to customers not seen in N months.
 3. **FAQ Builder** (customer_service) — interactive: owner answers prompts about common customer questions; agent drafts the FAQ for the KB. Closes the "knowledge base empty" loop at the source.
 4. **Seasonal Campaign** (marketing) — vertical-aware once packs ship; knows seasonal calendars (AC in summer, winterization in fall, etc.).
 5. **Pricing Question Handler** (customer_service) — specialised case of Customer Question; careful framing for "how much" questions. Prevents quoting prices the business profile doesn't authorise.
 6. **Page Editor** (marketing, if Hosted Business Page survives) — updates owner's hosted page copy via conversation.
 7. **Onboarding Coach** (system) — runs during signup wizard, asks vertical-specific questions to populate the business profile fully, reduces the KB-empty problem at the source.
 8. **Expense Tracker / Categorizer** (finance) — once a real billing integration ships, agent can categorize expenses or generate cost summaries.
 9. **Customer Birthday / Anniversary** (sales) — scheduled trigger; drafts a brief touch on key dates.
-

14. Build and verification checklist

For each agent in the library, before marking it "shipped":

- ☐ Conforms to the agent schema in §2
- ☐ Bucket assignment is correct
- ☐ Reads `business_profile` from shared context in every run
- ☐ Reasoning trace never reports a successful load of empty data
- ☐ Never emits `[bracketed placeholders]` for fields that exist in the profile
- ☐ Channel-appropriate output (no markdown in SMS drafts)
- ☐ Permission scope set, with sensible defaults
- ☐ At least 3 example interactions tested end-to-end
- ☐ Cost per run measured; abnormal costs investigated
- ☐ Internal team has run the agent at least 5 times in their own use
- ☐ One friendly external customer has used it once

When all checkboxes are green for every P1 + P2 agent, declare Phase 2 of the product plan complete and move into verification (Phase 3).

— end of v1 library —